A Survey of Python Type Analysis: Techniques, Challenges, and Emerging Directions

1st Mingyeong Jeong
Computer Science and Engineering
Chungnam National Univercity
Daejeon, South Korea
jmink002@o.cnu.ac.kr

2nd Sungho Lee
Computer Science and Engineering
Chungnam National Univercity
Daejeon, South Korea
eshaj@cnu.ac.kr

***Abstract*—The dynamic nature of Python poses fundamental challenges for accurate type inference, driving extensive research on type inference, checking, and recovery across static, dynamic, learning-based, and hybrid paradigms. This survey provides a comprehensive analysis of major Python type analysis efforts spanning static systems, dynamic monitoring frameworks, and emerging machine-learning-based models. We systematically identify and analyze representative studies collected through a keyword-driven search within the programming languages and software engineering domains, selecting those that explicitly address Python or its dynamic typing characteristics. This survey categorizes the Python type analysis landscape into four approaches: static analysis, dynamic monitoring, hybrid methods that combine static and runtime information, and AI-driven inference based on large language models. It provides a detailed review of the technical principles, advantages, limitations, and open challenges of each approach, with particular attention to Python-specific issues such as duck typing, flow-sensitive type changes, reflection, dynamic code generation, and heterogeneous containers. Our analysis shows that no single approach is sufficient to fully address Python type reasoning; instead, combining static, dynamic, and AI-based techniques in complementary ways is emerging as a promising direction for future research. By synthesizing existing research and tools, this survey clarifies the current landscape of Python type analysis and provides guidance for both practitioners and researchers seeking robust, practical, and forward-looking solutions.**

***Index Terms*—Python, Type analysis, Static typing, Dynamic typing, Hybrid, Machine learning**

I. INTRODUCTION

Python has experienced significant growth over the past decade and is now one of the most widely used programming languages worldwide. Its popularity has sharply increased since 2018 and it has consistently ranked as the most popular language in multiple surveys since 2022 [1]. This rise stems from Python's versatility across domains such as web development, data analytics, scientific computing, automation, and especially artificial intelligence and machine learning. The expansion of AI ecosystems—supported by deep learning frameworks (e.g., TensorFlow [2], PyTorch [3]) and natural language processing libraries (e.g., Hugging Face Transformers [4])—has further accelerated its adoption among both industry and research communities.

In addition to its broad applicability, Python's concise and natural-language-like syntax lowers the barrier to entry for new developers. Features such as dynamic typing, high-level data structures, and rich standard libraries allow programmers to express complex ideas with minimal boilerplate code and it allows rapid prototyping. As a result, Python has become a first-choice language for prototyping, experimentation, and educational purposes, fostering a vast ecosystem of open-source libraries and active developer communities.

However, Python's dynamic and interpreted nature introduces significant challenges for program reliability. Unlike compiled languages, where many errors can be caught at compile time, Python programs often surface errors only during execution. Among these runtime errors, type-related issues such as `TypeError` and `AttributeError` are particularly prevalent, accounting for a substantial portion of all reported bugs—studies suggest approximately 30% of runtime errors in Python stem from type mismatches [5]. This prevalence of type errors not only affects software reliability but also complicates maintenance and refactoring of large codebases, where inconsistent or unexpected type usage can propagate subtle bugs across modules [6].

Consequently, type checking and type analysis have become critical for improving the reliability, maintainability, and robustness of Python programs. The introduction of optional type annotations in Python 3.5 provided a mechanism for developers to express expected types explicitly, enabling static verification while preserving the language's inherent flexibility [7]. Tools such as `mypy` [8], `Pyre` [9], and `Pyright` [10] leverage these annotations to detect type inconsistencies before execution, reducing the likelihood of runtime failures. Beyond static checking, hybrid approaches combining static and runtime monitoring, as well as emerging AI-based techniques, have begun to address Python's dynamic and flexible behavior more effectively.

Despite these advancements, Python type analysis remains challenging due to several unique characteristics of the language. First, Python follows a form of structural typing, often referred to as "duck typing," where object compatibility is determined by available attributes and methods rather than nominal type hierarchies. Second, variable types in Python can change dynamically depending on the control flow, requiring type inference mechanisms to be flow-sensitive to achieve accuracy. Third, Python supports advanced dynamic features,

including reflection, runtime code generation through `eval` and `exec`, and dynamic module loading, all of which complicate static reasoning. Finally, Python data structures such as lists, dictionaries, and sets frequently contain heterogeneous elements whose types may vary over time, making traditional uniform-type assumptions insufficient. These language characteristics collectively create a significant gap between static type assumptions and the actual behavior of programs, motivating the development of specialized analysis techniques.

In this context, the present study aims to provide a comprehensive survey of Python type analysis techniques and tools, categorizing them according to their underlying methodologies and examining their strengths and limitations. Specifically, this paper addresses the following research questions (RQs):

- **RQ1** : How can Python's dynamic nature be preserved while ensuring program stability through type checking?
- **RQ2** : What are the main challenges in type inference and type analysis for Python programs?
- **RQ3** : What technical characteristics and limitations define existing Python type analysis tools?

To answer these questions, we analyze Python type analysis tools spanning several paradigms, including static analyzers leveraging type annotations, runtime and hybrid monitoring systems, and recently developed AI-powered methods enabled by large language models. By systematically examining these tools and approaches, we aim to clarify the current landscape of Python type analysis, highlight open challenges, and identify promising directions for future research.

In summary, this paper contributes a structured overview of Python type analysis, emphasizing the interplay between dynamic flexibility and the need for program reliability, and provides insights for both tool developers and researchers aiming to advance Python's type-checking ecosystem.

II. BACKGROUND

1) Python Type System: Python is a dynamically typed language in which variable types are determined at runtime rather than at the point of declaration. Because a variable's type is defined by the value assigned during execution, its type can change at any point while the program runs. Moreover, Python adopts a pure object-oriented design in which "everything is an object," making the notion of object compatibility a central issue in type analysis.

In dynamic languages such as Python, object compatibility is governed by duck typing. Under duck typing, compatibility is determined not by an object's nominal type (e.g., int, str, Parrot) but by whether it provides the required methods or attributes (such as fly() or len()).

Listing 1 illustrates this principle. As shown in lines 20–21, instances of Parrot and Airplane—despite having no inheritance or structural relationship—can be passed to the same function lift_off() without causing any errors. This demonstrates that compatibility is determined by the presence of the fly() method rather than by the concrete classes Parrot or Airplane. In contrast, when a Whale object, which does not define fly(), is passed to the same function (line 22), a TypeError is raised due to the missing attribute.

Listing 1 Example Python program demonstrating object compatibility determined by duck typing.

```
1  class Parrot:
2    def fly(self):
3      print("Parrot flying")
4
5  class Airplane:
6    def fly(self):
7      print("Airplane flying")
8
9  class Whale:
10   def swim(self):
11     print("Whale swimming")
12
13 def lift_off(entity):
14   entity.fly()
15
16 parrot = Parrot()
17 airplane = Airplane()
18 whale = Whale()
19
20 # prints `Parrot flying`
21 lift_off(parrot)
22
23 # prints `Airplane flying`
24 lift_off(airplane)
25
26 # Error : `'Whale' object has no attribute 'fly'`
27 lift_off(whale)
```

Thus, type determination in Python cannot rely solely on nominal types; it must also incorporate structural types defined by the methods and attributes available on an object. However, because these methods and attributes can be dynamically added, removed, or modified during execution, performing precise type analysis statically—without running the program—becomes inherently challenging.

A. Evolution of Python Type System

The ability to omit explicit types in Python allows developers to write concise code and rapidly prototype programs. However, this high degree of flexibility can reduce code readability, complicate maintenance, and increase the likelihood of unintended type-related bugs.

To mitigate these issues, Type Annotations were introduced in Python 3.5 for variables, functions, classes, and other constructs. Type Annotations are optional and do not affect the runtime behavior of a program, but they enable developers to explicitly state type information. This improves readability and developer productivity while serving as a form of contract that helps enhance program reliability, even though the annotations are not enforced at runtime.

Following the introduction of Type Annotations, various static type checkers have been actively developed in the Python ecosystem. Before this change, static analysis tools had to rely entirely on type inference, which is notoriously difficult in dynamic languages and often leads to imprecise analyses. With explicit annotations, static analyzers can perform significantly

Listing 2 Example Python program demonstrating object compatibility determined by duck typing.

```python
def foo(a: int, b: int):
    return a + b

# No Error
a = foo(1, 2)

# TypeError only in static type checker
b = foo(2, 3.14)

# TypeError in both static type checker and runtime
c = foo(3, "hello")
```

more accurate checks by leveraging developer-provided type information rather than depending solely on inference.

Python thus adopts a Gradual Typing paradigm that combines the advantages of both dynamic and static type systems. Under Gradual Typing, parts of the program where types can be determined statically are checked statically, while the rest continue to operate under the dynamic type system. This hybrid approach allows Python to achieve both flexibility and reliability in large-scale software development.

The concept of Gradual Typing can be illustrated with the Listing 2

In this example, the function foo explicitly specifies that both a and b should be of type int. This example highlights the following three key observations:

- foo(1, 2) satisfies the type specification, so neither the static type checker nor the runtime reports any errors.
- foo(2, 3.14) causes a type error in the static type checker because a float is passed where an int is expected. However, the Python runtime accepts the call and executes it, demonstrating that annotations are not enforced at runtime.
- foo(3, ”hello”) produces a type error in the static type checker, and additionally, it raises a runtime TypeError because Python cannot add an integer and a string.

This example illustrates how Python’s Gradual Typing performs static checking where type information is available, while maintaining dynamic typing behavior where static analysis is insufficient. This hybrid model provides the reliability of a static type system without sacrificing the flexibility of dynamic typing, making it suitable for large-scale software development.

III. RESEARCH RESULTS

In this paper, we systematically classify the major type analysis techniques used in Python, as illustrated in Figure 1, and compare their technical characteristics, strengths and weaknesses, as well as representative tools. Based on this analysis, we identify the fundamental limitations of existing approaches and propose future research directions to address these challenges.

A. *Select Papers*

In this survey, we aim to provide a comprehensive review of research on Python type analysis and type checking. To identify relevant studies, we conducted a systematic search of the literature using keywords such as “type analysis,” “static type checking,” and “dynamic type checking.” From the retrieved studies, we selected only those that focus on Python or explicitly address the dynamic characteristics of Python programs in their type inference or type checking techniques.

Based on these criteria, the survey includes traditional static type analyses [11], constraint-based approaches [12], machine learning–driven methods [13]–[15], and hybrid techniques combining static analysis and learning [16]–[18]. In addition, we also consider studies that aim to predict or repair Python runtime type errors [5], [19]–[21], covering research that extends beyond pure type inference to practical program improvement.

The selected studies collectively address diverse aspects of Python type analysis, including handling dynamic features, compound data types, and flow sensitivity. They represent a broad spectrum of methodologies—static, dynamic, and learning-based approaches—and are thus well-suited for a comprehensive comparison and discussion of the current state and future directions of Python type checking research.

B. *RQ1 : How can Python’s dynamic nature be preserved while ensuring program stability through type checking?*

Because Python is a dynamically typed language, the actual structure and state of objects are determined at runtime. While this provides significant flexibility, it also reduces the precision of static analysis and increases the likelihood of runtime errors such as TypeError and AttributeError. Consequently, researchers have proposed a wide range of analysis techniques that aim to improve program reliability through static or quasi-static methods without compromising Python’s dynamic nature. Table I shows the number of existing type checking approaches. These details are illustrated in the sections below.

TABLE I: Number of papers and tools classified by Type Checking Approaches (RQ1).

Static	Dynamic	Hybrid
21	2	5

*1) **Static Type Checking with Type Annotations**:* Since the introduction of type annotations, it has become possible to enhance program reliability using explicit static type information. Static type checkers such as Mypy [8] and Pyre [9] were developed on this basis, enabling the detection of potential TypeErrors in variables, function parameters, and return values before execution. Moreover, Python adopts a gradual typing approach, allowing static verification even when only parts of the program are annotated. This enables developers to strengthen type safety incrementally. Typically, type checking via static analysis verifies, as in Algorithm 1 whether the code

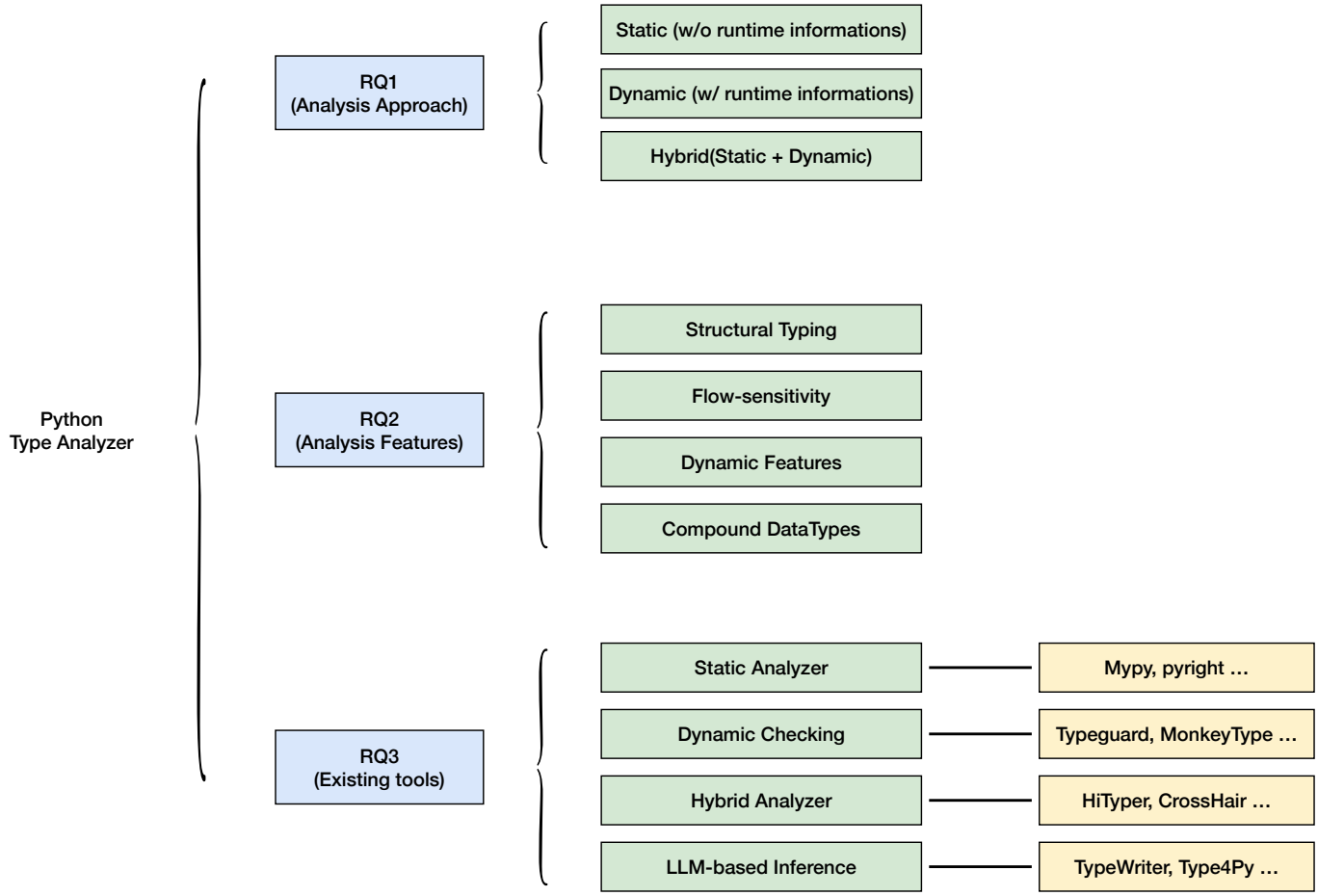

Fig. 1: Overview of Python type analysis in relation to the research questions (RQ1–RQ3).

matches the type annotations, and can further perform type inference to ensure that types are used correctly in practice.

However, because the analysis is limited to annotated regions, it cannot provide the same level of safety guarantees as fully statically typed languages. Furthermore, Python's type annotations rely primarily on nominal typing, which limits their ability to accurately capture Python's structural and behavior-based typing semantics, such as duck typing based on the presence of specific methods or attributes.

Algorithm 1 GradualTypeCheck

Require: Python function f with optional type annotations, arguments $args$
Ensure: TypeError if type mismatch detected

```
for each argument a_i in args do
    if f has type annotation for a_i then
        if type(a_i) ≠ annotation_type(a_i) then
            raise TypeError
        end if
    end if
end for
result = execute(f, args)
if f has return type annotation then
    if type(result) ≠ annotation_type(f.return) then
        raise TypeError
    end if
end if
return result
```

*2) **Dynamic Monitering and Runtime Enforcement**:* Dynamic type monitoring and runtime enforcement observe type-related constraints during the actual execution of a program and report violations immediately. Although this approach

cannot capture all errors prior to execution—as static analysis attempts to do—it provides high accuracy for type violations observed along concrete execution paths. In languages such as Python, where dynamic binding is pervasive, this approach is particularly practical because it can directly reproduce behavior-based runtime errors that static techniques often fail to detect.

The most widely used mechanism for runtime enforcement is decorator-based wrapping. By attaching a type-checking decorator to a function, as in Algorithm 2, the checker verifies at every call whether the argument and return values conform to the declared type annotations. Libraries such as typeguard [22] implement this strategy using typing.get_type_hints() and isinstance-based checks. This approach requires minimal modification to existing code and detects mis-typed values at the moment they occur, making it highly useful for debugging. However, because checks are performed on every call, runtime overhead is unavoidable, and complex type structures—such as generics or protocols—cannot always be fully verified at runtime.

Another form of dynamic monitoring leverages runtime traces to support subsequent analyses. MonkeyType [23], for example, records observed argument and return types during test execution and generates Mypy-compatible type annotations automatically. Because this approach reflects actual usage patterns, it is useful for introducing type hints into existing codebases or understanding complex functions. However, it depends heavily on test coverage and cannot infer types for branches that are never executed.

The strengths of dynamic type enforcement lie in its high precision for observed execution paths and the ease with which runtime errors can be reproduced. It can also be extended to verify a broad range of logical properties beyond types. Yet it also has inherent weaknesses: comprehensive safety requires sufficient test or execution coverage, runtime checking incurs performance overhead, and complex types such as generics or protocols may be difficult to validate from runtime information alone. Techniques such as conditional or sampling-based checks mitigate overhead but do not eliminate the fundamental coverage dependency.

In practice, a diverse ecosystem of dynamic type enforcement tools exists in Python. typeguard provides decorator-based runtime type checking, MonkeyType generates type hints from runtime traces, and CrossHair [24] combines dynamic execution with symbolic reasoning for hybrid verification. Collectively, these tools address classes of errors that are difficult to detect using purely static analysis in a highly dynamic language such as Python.

Algorithm 2 RuntimeTypeMonitor

Require: Python function f with optional type annotations
Ensure: Detect type violations during execution
Wrap f with a type-checking decorator
On each call to f:
for each argument a_i **do**
  if type annotation exists for a_i **then**
    if type(a_i) $\neq$ annotation_type(a_i) **then**
      Log type violation
    end if
  end if
end for
Execute original function f
if return type annotation exists **then**
  if type(return) $\neq$ annotation_type(return) **then**
    Log type violation
  end if
end if

*3) **Hybrid Approaches**:* To address both Python's dynamic nature and the inherent limitations of static analysis, recent research has increasingly focused on hybrid analysis, which integrates static and dynamic techniques. This approach combines the global reasoning capabilities of static analysis with the execution-based precision of dynamic analysis, providing more practical and robust guarantees for Python programs.

One representative hybrid strategy is the combination of gradual typing and runtime checking. For example, tools like Mypy and Pyre perform static verification only on code regions with explicit type annotations, leaving unannotated or ambiguous regions to be validated at runtime. This approach reduces unnecessary false positives during static analysis and ensures type safety at execution time, thereby improving overall accuracy. However, note that gradual typing-based approaches such as Mypy and Pyre are not included in the counting of hybrid methods in Table I.

The hybrid methods counted in Table I refer to approaches that combine two or more analysis techniques. For instance, they may leverage static analysis results to train AI models, or integrate dynamic execution data into static analysis to improve precision. This not only reduces overly conservative warnings produced by static analysis but also allows the abstraction level of the analysis to be refined incrementally with concrete runtime evidence. In essence, such integration enables bi-modal analysis, where static and dynamic information complement each other to achieve practical accuracy and reliability.

Hybrid techniques thus provide a compelling way to reflect both Python's dynamic nature and its real-world usage patterns. By combining the global perspective of static analysis with the precision of dynamic observations, they offer a level of type safety and tool usability that neither approach can achieve alone. Challenges remain—such as dependence on execution coverage, the cost of merging static and dynamic information, and runtime overhead due to dynamic checks—but hybrid analysis is widely regarded as one of the

most promising directions for Python type analysis, with recent research trends moving steadily toward deeper integration of the two paradigms.

*4) **RQ1 Summary: Landscape of Type Analysis Techniques for Python**:* Because Python is fundamentally a dynamically typed language, fully static type checking in the traditional sense is structurally impossible. Object attributes may be added or modified at runtime, and the return type of a function can vary depending on the execution path. Nevertheless, recent advances in type analysis techniques have made it possible to achieve a meaningful level of type safety even in Python.

Broadly, these efforts can be categorized into three directions. First, annotation-based static analysis performs ahead-of-time verification of program structure and enables early detection of errors. Second, runtime monitoring and checking techniques detect type violations along actual execution paths and can validate dynamic behaviors that are difficult to capture with static annotations alone. Third, hybrid approaches that integrate static and dynamic techniques supplement statically unpredictable regions with runtime checks and feed runtime observations back into the static analyzer to improve its precision, establishing a cyclic and complementary analysis workflow.

In summary, no single analysis technique can provide sufficient type safety for Python. However, by combining static type annotations, execution-based monitoring, and hybrid methodologies, it becomes possible to pursue both the flexibility of a dynamic language and the safety benefits of static reasoning. These complementary strategies form the modern landscape of type analysis for Python and serve as a foundation for future advances in this field.

Answer to RQ1

Recent work shows a clear shift from purely static analysis toward hybrid and runtime-assisted approaches for Python type checking. Static analyzers still play a central role in large-scale verification, but runtime monitoring complements them by capturing behaviors that static methods miss. As a result, hybrid systems have become the most practical way to balance Python's dynamic nature with meaningful type safety.

*C. **RQ2 : What are the main challenges in type inference and type analysis for Python programs?***

Type analysis in Python is fundamentally more challenging than in traditional statically typed languages due to its structural and dynamic characteristics. In particular, (1) structural typing–based object compatibility, (2) flow-sensitive types that change according to control flow, (3) advanced dynamic features such as reflection and dynamic code generation, and (4) the presence of heterogeneous containers are frequently cited as the primary obstacles to precise type analysis. Table II shows whether existing research and tools support these four key Python features. Each column corresponds to one of the four challenges, and the entries indicate whether a particular paper or tool provides support. This provides a clear overview of the strengths and limitations of current Python type analysis approaches.

First, Python follows a duck-typing model that is closer to structural typing than nominal typing. The meaning of an object is determined by the methods and attributes it provides, making it difficult to accurately model objects using only static type names. Most existing studies do not use structural typing; instead, they perform analysis based solely on type names (nominal typing). For this reason, current approaches cannot dynamically model Python's types, leading to high uncertainty in static type inference.

Second, the type of a Python variable may vary depending on the program's control flow. A single variable may assume different types across different branches, making it difficult for traditional static analysis to infer a consistent type. To address this, recent studies perform flow-sensitive analysis, but only about 50% of the works apply it; the rest rely on methods derived from static-typed languages that do not account for control flow. Flow-insensitive analysis sometimes uses techniques such as Static Single Assignment (SSA) to represent control flow. However, increasing precision using these methods rapidly raises computational costs. Therefore, flow-aware analysis is essential, and balancing accuracy with performance remains a central challenge in Python type analysis.

Third, Python's powerful dynamic features—reflection and dynamic code generation—fundamentally threaten the soundness of any static type analysis. Dynamic attribute access via getattr and __getattribute__, runtime code generation using eval() and exec(), dynamic module loading via importlib, and metaclass-based class generation all defer program structure determination until runtime. Accurately analyzing these features would require nearly full program execution, making precise static type inference difficult within traditional frameworks. However, as shown in Table II, about 80% of studies support some dynamic features; this is a generous count that includes partial support. Full support for all dynamic features is not achievable in static analysis except through execution-based approaches.

Finally, Python's lists, sets, and dictionaries inherently allow heterogeneous elements. A single list may contain integers, strings, and user-defined objects, and element types may evolve during execution. Simple uniform type models are insufficient, and more sophisticated representations—such as union types, intersection types, or lattice-based type spaces—are required. All surveyed studies and tools account for this feature, highlighting that heterogeneous containers are a critically important aspect of Python type analysis.

TABLE II: RQ2: Table showing whether existing Python type analysis papers and tools support key Python features, including structural typing, flow-sensitive types, dynamic features, and compound data types. (✓ is support, × is not.)

Paper/Tool	Structural Typing	Flow-sensitivity	Dynamic Features	Compound DataTypes
[11]	×	×	×	✓
[25]	×	×	×	✓
[12]	×	✓	×	✓
[26]	×	×	✓	✓
[27]	×	×	✓	✓
[28]	×	×	✓	✓
[14]	×	×	✓	✓
[19]	×	✓	✓	✓
[29]	×	✓	✓	✓
[16]	×	✓	✓	✓
[30]	×	×	✓	✓
[13]	×	×	✓	✓
[17]	×	×	✓	✓
[20]	×	✓	✓	✓
[18]	×	✓	✓	✓
[31]	×	✓	×	✓
[15]	×	×	✓	✓
[5]	✓	✓	✓	✓
[21]	✓	✓	✓	✓
[23]	×	×	✓	✓
[24]	×	×	✓	✓
[22]	×	×	✓	✓
[32]	✓	✓	✓	✓
[8]	×	×	×	✓
[9]	×	✓	✓	✓
[33]	×	✓	✓	✓
[10]	×	✓	✓	✓
27	3	13	22	27

Answer to RQ2

Python type analysis exhibits high complexity due to (1) structural typing–based non-uniformity, (2) types that change continually depending on control flow, (3) the possibility of runtime code generation, and (4) heterogeneous containers. These characteristics make it difficult for traditional static analysis frameworks to achieve accurate and sound modeling, highlighting the need for new analysis paradigms tailored to Python's inherently dynamic nature.

*D. **RQ3 : What technical characteristics and limitations define existing Python type analysis tools?***

The Python ecosystem includes a wide range of type analysis tools, spanning static, dynamic, hybrid, and, more recently, LLM-based approaches. Each category adopts a distinct technical strategy to handle Python's highly dynamic language characteristics, resulting in a mix of complementary strengths and inherent limitations. This section classifies major tools into four categories and analyzes their technical features and constraints.

Static type checkers such as Mypy, Pyright, and Pyre are the most widely used tools in practice. These systems construct a type environment primarily from user-provided type annotations and verify type consistency through constraint solving. They support advanced features of Python's typing system—such as Protocols and TypedDicts—to enable structural subtyping and partial type inference. This design yields fast analysis performance and reasonable precision for large-scale codebases, making static analysis highly practical. However, their heavy reliance on annotations means that accuracy degrades sharply in real-world industrial code, where annotations are often missing or incomplete. They also cannot fully model Python's dynamic features such as reflection, monkey patching, and dynamic attribute creation, resulting in unavoidable false positives and negatives.

In contrast, dynamic and hybrid analysis tools leverage runtime information to achieve higher accuracy. Tools such as Typeguard and contract-based libraries validate arguments, return values, and structural properties of objects at execution time, allowing detection of type violations soundly during actual runs. Tools like MonkeyType collect type hints from execution traces, enabling high-precision, test-driven type inference. These approaches require little to no annotation but structurally depend on test coverage; unexecuted paths cannot be analyzed. Runtime instrumentation also introduces performance overhead. Recent academic work explores advanced static techniques, including constraint-based analysis, symbolic execution, and SSA-based type analysis, employing refined type models such as union-type lattices, intersection types, and path-sensitive constraint solving to achieve higher

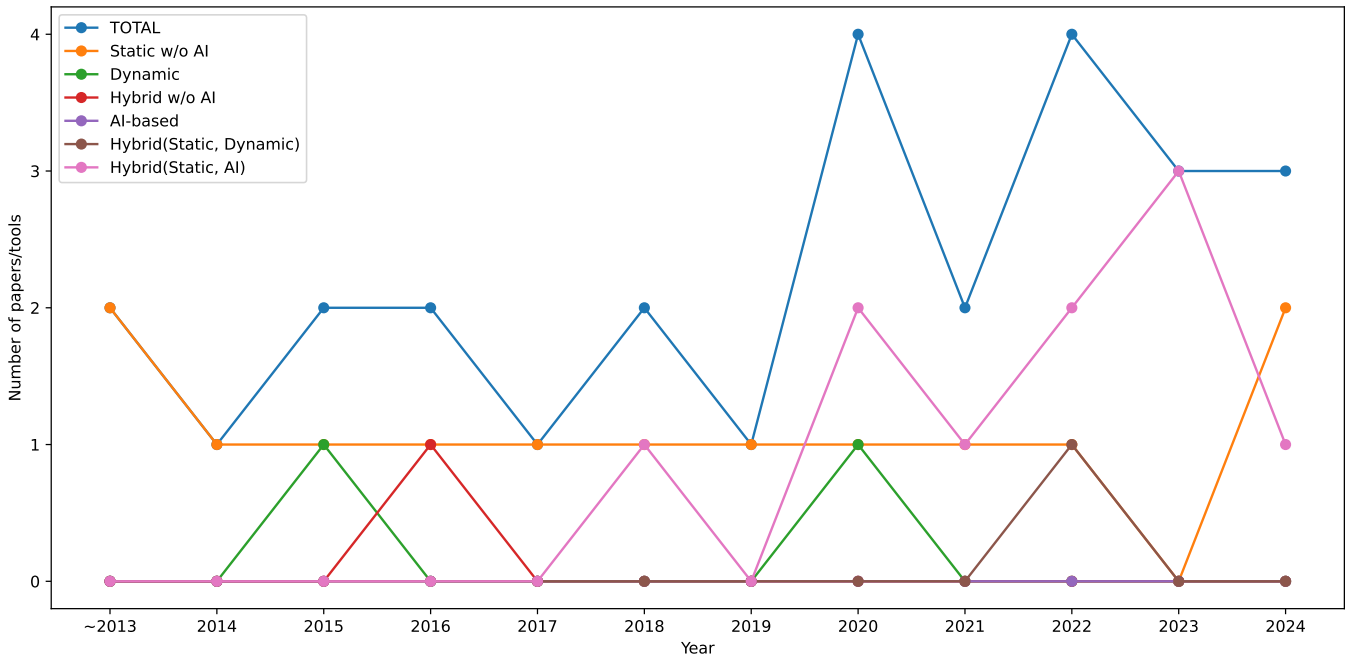

Fig. 2: Number of Python type analysis papers and tools developed per year (2013–2024) by methodology. This line chart illustrates the annual trends in the development of research and tools across different type checking approaches, including static, dynamic, hybrid, and AI-based methods.

precision [21]. However, these techniques are computationally expensive and difficult to apply to real-world codebases, especially in the presence of Python's irregular dynamic features.

Finally, LLM-based type analysis—using models like GPT or CodeLlama—has emerged as a new direction. LLMs can infer types with high coverage even for minimally annotated codebases, and perform semantic-level reasoning to detect errors that traditional static analyzers miss. However, LLMs operate as black-box models; their reasoning processes are opaque, subject to hallucination, and fundamentally unable to guarantee soundness.

The changing trends in Python type checking research can be observed in Figure 2. Over the years, the use of LLM-based approaches has gradually increased, and notably, hybrid methodologies that combine static analysis with AI-based techniques to ensure type soundness are being actively explored.

In summary, Python type analysis tools exhibit complementary strengths depending on their underlying analysis paradigm, but no single approach can fully capture the complexity of Python's dynamic semantics. Static analysis is efficient yet incomplete, dynamic analysis is precise yet coverage- and cost-constrained, and AI-based methods are flexible but lack soundness. Consequently, recent research trends increasingly favor hybrid approaches that combine static, dynamic, and AI-based techniques. This observation underscores that Python type analysis is not a problem solvable by any single method.

Answer to RQ3

Python type analysis tools can be classified into static, dynamic, research-oriented, and LLM-based approaches. Static analysis is fast but struggles with dynamic features, while dynamic and hybrid methods offer accuracy at the cost of coverage and runtime overhead. LLM-based approaches enable semantic-level reasoning but lack determinism and soundness. Overall, no single technique is sufficient, and hybrid combinations of static, dynamic, and AI-based methods are becoming increasingly important.

IV. DISCUSSION

This paper systematically reviews the Python type analysis ecosystem and compares how static, dynamic, and AI-based approaches address specific challenges while highlighting their limitations. Through this, the impact of Python's dynamic features on the design of analysis techniques becomes evident.

As observed in RQ1, achieving program safety with a single static type system in Python is challenging. Consequently, static type checking, runtime enforcement, and hybrid approaches coexist, forming a unique technical landscape. This layered strategy represents a practical compromise, balancing developer productivity and dynamic flexibility while ensuring a certain level of reliability and error detection.

The structural challenges highlighted in RQ2 introduce problems that cannot be fully addressed by traditional language theory. Specifically, structural typing, flow-sensitive

type changes, heterogeneous containers, and dynamic code generation via reflection or eval fundamentally challenge assumptions of conventional static analysis (e.g., nominal hierarchy, stable object model, closed world assumption). These issues demand reconsideration of the analysis model itself. For instance, flow-sensitive analyses provide high precision but incur high computational costs, whereas flow-insensitive approaches scale better but with limited precision. This underscores the constant trade-offs Python analyzers must navigate among precision, scalability, and soundness.

A similar trend emerges in the tool comparison of RQ3. Static analyzers remain essential for large-scale codebases, yet they cannot fully model Python's dynamic features. Dynamic and hybrid tools improve accuracy using runtime information but cannot guarantee fundamental completeness due to coverage limitations. Research-oriented tools leveraging constraint solving or symbolic execution offer high precision but are limited in practical adoption due to computational cost and challenges in handling dynamic code. Recently, LLM-based analyses provide high inference coverage even in low-annotation settings but cannot ensure soundness due to non-determinism and hallucinations.

Overall, the central challenge in Python type analysis is the discontinuity between the language's dynamic, open-ended nature and the demands of static safety. Each approach provides strategies to bridge this gap, yet no single method offers a complete solution. Hybrid or feedback-driven approaches that integrate static, dynamic, and AI-based techniques are increasingly recognized as a promising direction for future development.

V. CONCLUSION

This survey systematically reviewed the technical trends and research directions in Python type analysis. Python's dynamic execution model fundamentally conflicts with the fixed and closed type structures assumed by traditional static analyses, making it challenging to build practical analyzers by simply adapting type analysis techniques from other languages. Consequently, the Python ecosystem has seen the parallel development of a wide range of strategies, including static analyzers, runtime monitoring systems, execution trace-based type inference, symbolic execution, constraint solving, and more recently, LLM-based approaches.

From this analysis, several conclusions can be drawn:

- Static analysis alone cannot fully capture Python's dynamic features. Structural typing, dynamic attributes, reflection, and heterogeneous containers violate assumptions of conventional static type theory.
- Dynamic and hybrid approaches provide high accuracy but cannot fully resolve coverage and performance limitations.
- LLM-based approaches offer new possibilities but still fail to guarantee soundness.

Therefore, a hybrid, multi-layered approach that combines the scalability of static analysis, the precision of dynamic analysis, and the generalization capability of AI-based inference is essential in Python. Overall, Python type analysis remains an evolving domain, situated in the tension between the language's dynamic design philosophy and the requirements of static safety, with each technique playing a complementary role in bridging this gap. Future research is expected to formalize the interaction between static, dynamic, and AI-based techniques, exploring new analysis models that balance soundness, precision, and scalability.

REFERENCES

[1] Tiobe index: The python programming language, https://www.tiobe.com/tiobe-index/python/.

[2] Tensorflow: An end-to-end platform for machine learning, https://www.tensorflow.org/.

[3] Pytorch, https://pytorch.org/.

[4] Hugging face-transformers, https://huggingface.co/docs/transformers/index.

[5] W. Oh, H. Oh, Towards effective static type-error detection for python, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 1808–1820. doi:10.1145/3691620.3695545.
URL https://doi.org/10.1145/3691620.3695545

[6] F. K. et al., An empirical study of type-related defects in python projects, IEEE Transactions on Software Engineering (2021).

[7] Pep 484 - type hints, https://peps.python.org/pep-0484/.

[8] Mypy: Static typing for python, https://github.com/python/mypy.

[9] pyre-check: Performant type-checking for python, https://github.com/facebook/pyre-check.

[10] pyright: Static type checker for python, https://github.com/microsoft/pyright.

[11] T. Dong, L. Chen, Z. Xu, B. Yu, Static type analysis for python, in: 2014 11th Web Information System and Application Conference, 2014, pp. 65–68. doi:10.1109/WISA.2014.20.

[12] M. Hassan, C. Urban, M. Eilers, P. Müller, Maxsmt-based type inference for python 3, in: H. Chockler, G. Weissenbacher (Eds.), Computer Aided Verification, Springer International Publishing, Cham, 2018, pp. 12–19.

[13] S. Cui, G. Zhao, Z. Dai, L. Wang, R. Huang, J. Huang, Pyinfer: Deep learning semantic type inference for python variables, CoRR abs/2106.14316 (2021). arXiv:2106.14316.
URL https://arxiv.org/abs/2106.14316

[14] J. Wei, M. Goyal, G. Durrett, I. Dillig, Lambdanet: Probabilistic type inference using graph neural networks, CoRR abs/2005.02161 (2020). arXiv:2005.02161.
URL https://arxiv.org/abs/2005.02161

[15] J. Wei, G. Durrett, I. Dillig, Typet5: Seq2seq type inference using static analysis, in: The Eleventh International Conference on Learning Representations, 2023.
URL https://openreview.net/forum?id=4TyNEhI2GdN

[16] Y. Peng, C. Gao, Z. Li, B. Gao, D. Lo, Q. Zhang, M. Lyu, Static inference meets deep learning: a hybrid type inference approach for python, in: Proceedings of the 44th International Conference on Software Engineering, ICSE '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 2019–2030. doi:10.1145/3510003.3510038.
URL https://doi.org/10.1145/3510003.3510038

[17] K. Sun, Y. Zhao, D. Hao, L. Zhang, Static type recommendation for python, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, ASE '22, Association for Computing Machinery, New York, NY, USA, 2023. doi:10.1145/3551349.3561150.
URL https://doi.org/10.1145/3551349.3561150

[18] Y. Yan, Y. Feng, H. Fan, B. Xu, Dlinfer: Deep learning with static slicing for python type inference, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023, pp. 2009–2021. doi:10.1109/ICSE48619.2023.00170.

[19] W. Oh, H. Oh, Pyter: effective program repair for python type errors, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Association for Computing Machinery, New York,

NY, USA, 2022, p. 922–934. doi:10.1145/3540250.3549130.
URL https://doi.org/10.1145/3540250.3549130

[20] Y. W. Chow, L. Di Grazia, M. Pradel, Pyty: Repairing static type errors in python, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, Association for Computing Machinery, New York, NY, USA, 2024. doi:10.1145/3597503.3639184.
URL https://doi.org/10.1145/3597503.3639184

[21] J. Wu, C. Lemieux, Quac: Quick attribute-centric type inference for python, Proc. ACM Program. Lang. 8 (OOPSLA2) (Oct. 2024). doi:10.1145/3689783.
URL https://doi.org/10.1145/3689783

[22] typeguard: Run-time type checker for python, https://github.com/agronholm/typeguard.

[23] Monkeytype: A python library that generates static type annotations by collecting runtime types, https://github.com/Instagram/MonkeyType.

[24] Crosshair: An analysis tool for python that blurs the line between testing and type systems., https://github.com/pschanely/CrossHair.

[25] F. Ortin, J. B. G. Perez-Schofield, J. M. Redondo, Towards a static type checker for python, in: European Conference on Object-Oriented Programming (ECOOP), Scripts to Programs Workshop, STOP, Vol. 15, 2015, pp. 1–2.

[26] Z. Xu, X. Zhang, L. Chen, K. Pei, B. Xu, Python probabilistic type inference with natural language support, in: Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2016, Association for Computing Machinery, New York, NY, USA, 2016, p. 607–618. doi:10.1145/2950290.2950343.
URL https://doi.org/10.1145/2950290.2950343

[27] V. J. Hellendoorn, C. Bird, E. T. Barr, M. Allamanis, Deep learning type inference, in: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2018, Association for Computing Machinery, New York, NY, USA, 2018, p. 152–162. doi:10.1145/3236024.3236051.
URL https://doi.org/10.1145/3236024.3236051

[28] M. Pradel, G. Gousios, J. Liu, S. Chandra, Typewriter: neural type prediction with search-based validation, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2020, Association for Computing Machinery, New York, NY, USA, 2020, p. 209–220. doi:10.1145/3368089.3409715.
URL https://doi.org/10.1145/3368089.3409715

[29] M. Hu, Y. Zhang, W. Huang, Y. Xiong, Static type inference for foreign functions of python, in: 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE), 2021, pp. 423–433. doi:10.1109/ISSRE52982.2021.00051.

[30] A. M. Mir, E. Latoškinas, S. Proksch, G. Gousios, Type4py: practical deep similarity learning-based type inference for python, in: Proceedings of the 44th International Conference on Software Engineering, ICSE '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 2241–2252. doi:10.1145/3510003.3510124.
URL https://doi.org/10.1145/3510003.3510124

[31] J. Elkobi, B. Gruner, T. Sonnekalb, C.-A. Brust, Tipical - type inference for python in critical accuracy level, in: 2023 IEEE/ACIS 21st International Conference on Software Engineering Research, Management and Applications (SERA), 2023, pp. 16–21. doi:10.1109/SERA57763.2023.10197800.

[32] R. Monat, A. Ouadjaout, A. Miné, Static Type Analysis by Abstract Interpretation of Python Programs, in: R. Hirschfeld, T. Pape (Eds.), 34th European Conference on Object-Oriented Programming (ECOOP 2020), Vol. 166 of Leibniz International Proceedings in Informatics (LIPIcs), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2020, pp. 17:1–17:29. doi:10.4230/LIPIcs.ECOOP.2020.17.
URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2020.17

[33] pytype: A static type analyzer for python code, https://github.com/google/pytype.